

```
diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
def parse_value_to_Float(val):
-   if isinstance(val, (int, float)):
-       return float(val)
-   if not val or val == "N/A" or val == "0":
-       return 0.0
-   val = str(val).strip()
-   if val.upper() in ("N/A", "NA", "NONE", "NULL", "NaN", "--") or
val.startswith("Loading"):
-       return 0.0
-   is_pct = "%" in val
-   val = val.replace(" ", "").replace("-", "").replace("%", "").replace("X", "")
-
-   # Normalize common Coinglass/TV glyphs
-   val = {
-       val.replace("0", "0"),
-       .replace("-", "-"),
-       .replace("u2122", "-"),
-       .strip()
-   }
-
-   if " " in val:
-       val = val.split()[0]
```

```

multiplier = 1.0
+ if val.upper().endswith("K"):
+     multiplier = 1000.0
+     val = val[:-1]
+ elif val.upper().endswith("M"):
+     multiplier = 1000000.0
+     val = val[:-1]
+ elif val.upper().endswith("B"):
+     multiplier = 1000000000.0
+     val = val[:-1]
+ elif val.upper().endswith("T"):
+     multiplier = 1000000000000.0
+     val = val[:-1]
+
+ try:
+     parsed_val = float(val) * multiplier
+     if is_pct:
+         return parsed_val / 100.0
+     return parsed_val
+ except Exception:
+     return 0.0
+
+ """Parse CoinGlass/TV numeric strings.
+
+ Important: this function returns 0.0 for unparseable values for backward
+ compatibility. Use parse_optional_value() when missing-vs-zero matters.
+ """
+
+ if isinstance(val, (int, float)) and not isinstance(val, bool):
+     try:
+         f = float(val)
+         return f if math.isfinite(f) else 0.0
+     except Exception:
+         return 0.0
+ if val is None:
+     return 0.0
+
+ s = str(val).strip()
+ if (
+     not s
+     or s.upper() in ("N/A", "NA", "NONE", "NULL", "NaN", "...")
+     or s.startswith("Loading")
+ ):
+     return 0.0
+
+ s = (
+     s.replace("\u2212", "-")
+     .replace("\u2012", "-")
+     .replace("\u2013", "-")
+     .replace("\u2014", "-")
+     .replace(",", "")
+     .replace("$", "")
+ )
+
+ # Extract the first numeric digit after removing all spaces. This
+ # avoids turning "123 456" into "123456".
+ m = re.search(
+     r"([+-]?(\d+(\.\d*)?|\.\d+)(?:[eE][+-]?\d+)?\s*[KMmbBtP]?s*%?)",
+     s,
+ )
+ if not m:
+     return 0.0
+
+ token = m.group(1).strip()
+ is_pct = token.endswith("K")
+ token = token.replace("K", "").replace(" ", "")
+
+ multiplier = 1.0
+ if token[-1].upper() == "K":
+     multiplier = 1_000.0
+     token = token[:-1]
+ elif token[-1].upper() == "M":
+     multiplier = 1_000_000.0
+     token = token[:-1]
+ elif token[-1].upper() == "B":
+     multiplier = 1_000_000_000.0
+     token = token[:-1]
+ elif token[-1].upper() == "T":
+     multiplier = 1_000_000_000_000.0
+     token = token[:-1]
+
+ try:
+     out = float(token) * multiplier
+     return out / 100.0 if is_pct else out
+ except Exception:
+     return 0.0
+
+
+def parse_optional_value(val):
+    """Return None for genuinely missing values, preserving real numeric zero."""
+    if val is None:
+        return None
+    if isinstance(val, str):
+        s = val.strip()
+        if (
+            not s
+            or s.upper() in ("N/A", "NA", "NONE", "NULL", "NaN", "...")
+            or s.startswith("Loading")
+        ):
+            return None
+    return parse_value_to_float(val)
+
+
+def normalize_display_value(val):
+    if val is None:
+        return "N/A"
+    s = str(val).strip()
+    if not s:
+        return "N/A"
+    return (
+        s.replace("\u2212", "-")
+        .replace("\u2012", "-")
+        .replace("\u2013", "-")
+        .replace("\u2014", "-")
+        .strip()
+    )
+
+
+@@
+class UnifiedDataFeed:
+    def __init__(self):
+        self.HISTORY_MAXLEN = 14410
+        self.latest_data = collections.defaultdict(dict)
+        self.history = collections.defaultdict(lambda: collections.defaultdict(lambda:
+collections.deque(maxlen=14410)))
+        self.history = collections.defaultdict(
+            lambda: collections.defaultdict(lambda:
+collections.deque(maxlen=self.HISTORY_MAXLEN))
+        )
+        self.last_update[symbol][metric] = epoch seconds of last accepted non-missing update.
+        self.latest_update = collections.defaultdict(dict)
+        self.TRACKED_HIST_KEYS = frozenset(("price", "oi", "funding", "dwd", "spot_cwd",
+        "future_cwd", "liquidations_long", "liquidations_short", "rsi", "is_ratio", "cb_premium",
+        "delta", "bid_vol", "ask_vol", "poc"))
+        self.lock = threading.RLock()
+
+
+    @staticmethod
+    def _base_hist_key(key: str) -> str:
+        return re.sub(r"([15m|30m|1h|4h|1d])$", "", key or "")
+
+    def _should_track_history(self, key: str) -> bool:
+        return self._base_hist_key(key) in self.TRACKED_HIST_KEYS
+
+
+    @staticmethod

```

```

+ def _is_missing_update(v) -> bool:
+     if v is None:
+         return True
+     if isinstance(v, str):
+         s = v.strip()
+         return (
+             not s
+             or s.upper() in ("N/A", "NA", "NONE", "NULL", "NaN", "...")
+             or s.startswith("Loading")
+         )
+     return False
+
+ @
+ def _populate_key_history(self, symbol, feed_key, hist_vals, now, source_interval=300):
+     self.history[symbol][feed_key] = []
+     clean_vals = []
+     for x in hist_vals or []:
+         parsed = parse_optional_value(x)
+         if parsed is not None and math.isfinite(parsed):
+             clean_vals.append(parsed)
+     hist_vals = clean_vals
+     n = len(hist_vals)
+     target_interval = 30
+     target = collections.deque(maxlen=self.HISTORY_MAXLEN)
+
+     if n == 0:
+         return
+     elif n == 1:
+         target_length = 480
+         for k in range(target_length):
+             t = now - (target_length - 1 - k) * target_interval
+             self.history[symbol][feed_key].append((t, hist_vals[0]))
+             target.append((t, hist_vals[0]))
+     else:
+         target_length = int((n - 1) * source_interval / target_interval) + 1
+
+     @
+     self.history[symbol][feed_key].append((t, val))
+     target.append((t, val))
+
+     self.history[symbol][feed_key] = target
+     self.last_update[symbol][feed_key] = now
+
+     @
+     def add_to_history(self, symbol, key, val):
+         # Called under lock in update_from_layout
+         if not self._should_track_history(key):
+             return
+         parsed = parse_optional_value(val)
+         if parsed is None or not math.isfinite(parsed):
+             return
+
+         # 30-second buckets give ~10 points per 5-min window for meaningful z-scores
+         BUCKET_SECONDS = 30
+         bucket_t = int(time.time() / BUCKET_SECONDS) * BUCKET_SECONDS
+         now = time.time()
+         bucket_t = int(now / BUCKET_SECONDS) * BUCKET_SECONDS
+         hist = self.history[symbol][key]
+         if not isinstance(hist, collections.deque):
+             hist = collections.deque(list(hist)[-self.HISTORY_MAXLEN:],
+                                     maxlen=self.HISTORY_MAXLEN)
+         self.history[symbol][key] = hist
+         if not hist or hist[-1][0] < bucket_t:
+             hist.append((bucket_t, val))
+             hist.append((bucket_t, parsed))
+         else:
+             hist[-1] = (bucket_t, val)
+
+         # trim history to last 120 hours (lookback-friendly) using slice deletion for O(n)
+         bulk prune
+         # deque handles trimming
+         hist[-1] = (bucket_t, parsed)
+         self.last_update[symbol][key] = now
+         self.latest_data[symbol]["last_update"] = now
+
+     @
+     def update_from_layout(self, layout_data):
+         with self.lock:
+             for k, v in data.items():
+                 if k.endswith('_history'):
+                     continue
+                 if symbol in ["XAUUSD", "XAGUSD", "CLUSD", "NATGASUSD"] and k ==
+                 "spot_cvd_15m":
+                     self.latest_data[symbol][k] = 'N/A'
+                     continue
+                 if self._is_missing_update(v):
+                     continue
+                 if k == 'liquidations_long_15m' and (v is None or v <= 0.0):
+                     continue
+                 if k == 'liquidations_short_15m' and v is None:
+                     continue
+                 if k in ['price', 'oi'] and (v is None or v <= 0.0):
+                     continue
+                 if symbol in ["XAUUSD", "XAGUSD", "CLUSD", "NATGASUSD"] and k ==
+                 "spot_cvd_15m":
+                     self.latest_data[symbol][k] = 'N/A'
+                 else:
+                     self.latest_data[symbol][k] = v
+                 self.last_update[symbol][k] = time.time()
+                 self.add_to_history(symbol, k, self.latest_data[symbol][k])
+                 if k == 'ls_ratio':
+                     self.latest_data[symbol]['macro_long_short_ratio'] = v
+
+     @
+     def update_from_footprint(self):
+         with self.lock:
+             with footprint_store_lock:
+                 for symbol, data in footprint_store.items():
+                     # Snapshot under footprint lock, then release it before taking feed.lock.
+                     # This avoids nested lock contention with websocket/profile workers.
+                     with footprint_store_lock:
+                         fp_snapshot = {s: d.copy() for s, d in footprint_store.items()}
+
+                     with self.lock:
+                         now = time.time()
+                         for symbol, data in fp_snapshot.items():
+                             price = data.get('price', 0.0)
+                             last_upd = data.get('last_update', 0.0)
+
+                 @
+                 self.latest_data[symbol]['delta'] = data.get('delta', 0.0)
+                 self.latest_data[symbol]['bid_vol'] = data.get('bid_vol', 0.0)
+                 self.latest_data[symbol]['ask_vol'] = data.get('ask_vol', 0.0)
+                 self.latest_data[symbol]['poc'] = data.get('poc', 'N/A')
+
+                 @
+                 self.latest_data[symbol]['of_aborption_top'] =
+                 data.get('of_aborption_top', 1.0)
+                 self.latest_data[symbol]['of_delta_ratio'] = data.get('delta_percent',
+                 0.0) / 100.0
+
+                 for hist_key in ("delta", "bid_vol", "ask_vol"):
+                     self.add_to_history(symbol, hist_key,
+                     self.latest_data[symbol].get(hist_key))
+                 self.last_update[symbol]['footprint'] = now
+
+                 def metric_age(self, symbol: str, key: str) -> float:
+                     with self.lock:
+                         ts = self.last_update.get(symbol, {}).get(key, 0.0)
+                         return float("inf") if ts <= 0 else time.time() - ts
+
+                 def is_symbol_stale(self, symbol: str, keys=("price",), max_age: float = 180.0) ->
+                 bool:
+                     with self.lock:
+                         sym_updates = self.last_update.get(symbol, {})
+                         for key in keys:
+                             if sym_updates.get(key, 0.0) <= 0:

```

```
+         return True
+         if time.time() - sym_updates[key] > max_age:
+             return True
+     return False
```

Copy

Patch 2 — Fix invalid JS extractor syntax and broken display cleaning

```
diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
- const numRe = /[a-]?(d(\.d)?)([0Bb])?$/;
+ const minusRe = /(\u2012\u2012\u2013\u2014)/g;
+ const numRe = /[a-]?(d(\.d)?)([0Bb])?$/;
@@
- const t = lines[i].replace(/g, '-').trim();
+ const t = lines[i].replace(minusRe, '-').trim();
+
@@
- return lines[lines.length - 1].replace(/g, '-').trim();
+ return lines[lines.length - 1].replace(minusRe, '-').trim();
@@
- out.liquidations = lines.filter(l => numRe.test(l.replace(/g, '')) ||
l.includes('0')).join(' ').replace(/g, '');
+ out.liquidations = lines
+ .map(l => l.replace(minusRe, '-'))
+ .filter(l => numRe.test(l) || l.includes('0'))
+ .join(' ');
@@
- out.price = v[k].innerText.trim().replace(/g, '');
+ out.price = v[k].innerText.trim().replace(minusRe, '-');
@@
- if (m) out.price = m[1].replace(/g, '');
+ if (m) out.price = m[1].replace(minusRe, '-');
-
- def clean_val(val):
-     if val is None: return "N/A"
-     return str(val).replace("", "0").replace("", "-").strip()
+ def clean_val(val):
+     return normalize_display_value(val)
@@
- def clean_val(val):
-     if val is None: return "N/A"
-     return str(val).replace("", "0").replace("", "-").strip()
+ def clean_val(val):
+     return normalize_display_value(val)
@@
MULTI_FRAME_EXTRACTION_JS = r'''() => {
    let result = {};
@@
- let data = {};
+ let data = {};
+
+ let minusRe = /(\u2012\u2012\u2013\u2014)/g;
@@
- data.price = valValues[k].innerText.trim().replace(/g, '-');
+ data.price = valValues[k].innerText.trim().replace(minusRe, '-');
});
@@
SINGLE_FRAME_EXTRACTION_JS = r'''() => {
    let doc = document;
    let data = {};
+ let minusRe = /(\u2012\u2012\u2013\u2014)/g;
@@
- data.price = valValues[k].innerText.trim().replace(/g, '-');
+ data.price = valValues[k].innerText.trim().replace(minusRe, '-');
});
@@
def parse_value(self, text, pattern):
    if not text:
        return "N/A"
-     if "" in text:
-         return "0"
    match = re.search(pattern, text, re.IGNORECASE)
    if match:
        val = match.group(1)
-         if val == "":
-             return "0"
        return val.replace("", "-").strip()
+     return normalize_display_value(val)
    return "N/A"
```

Copy

Patch 3 — Prevent partial DOM failures from rolling zeros into history

```
diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
def get_page_live_data(self, page, page_idx):
@@
    for win_idx, target_symbol in enumerate(page_assets, start=1):
        symbol_key = target_symbol.upper()
        win_key = f"win{win_idx}"
        win_dom = dom_results.get(win_key, {})
        dom_data = win_dom.get("data", {}) if win_dom.get("success") else {}
+
+     # Do not convert a failed/empty DOM scrape into zeros. Missing data
+     # must leave the previous feed value intact and become stale via
+     # UnifiedDataFeed.last_update.
+     if not win_dom.get("success") or not dom_data:
+         continue
+
-     price_val = parse_value_to_float(dom_data.get("price", 0.0))
-     spot_cvd_val = parse_value_to_float(dom_data.get("spot_cvd", 0.0))
-     futures_cvd_val = parse_value_to_float(dom_data.get("futures_cvd", 0.0))
-     funding_val = parse_value_to_float(dom_data.get("funding_rate", 0.0))
-     oi_val = parse_value_to_float(dom_data.get("open_interest", 0.0))
-     cb_premium_val = parse_value_to_float(dom_data.get("premium_index", 0.0))
+     price_val = parse_optional_value(dom_data.get("price"))
+     spot_cvd_val = parse_optional_value(dom_data.get("spot_cvd"))
+     futures_cvd_val = parse_optional_value(dom_data.get("futures_cvd"))
+     funding_val = parse_optional_value(dom_data.get("funding_rate"))
+     oi_val = parse_optional_value(dom_data.get("open_interest"))
+     cb_premium_val = parse_optional_value(dom_data.get("premium_index"))
+
-     parsed_rsi = parse_value_to_float(dom_data.get("rsi", 0.0))
-     rsi_val = parsed_rsi if parsed_rsi > 0 else ("N/A" if dom_data.get("rsi",
'N/A') == 'N/A' else 50.0)
+     parsed_rsi = parse_optional_value(dom_data.get("rsi"))
+     rsi_val = parsed_rsi if parsed_rsi is not None and parsed_rsi > 0 else None
+
-     parsed_ls = parse_value_to_float(dom_data.get("ls_ratio", 0.0))
-     ls_val = parsed_ls if parsed_ls > 0 else 1.0
+     parsed_ls = parse_optional_value(dom_data.get("ls_ratio"))
+     ls_val = parsed_ls if parsed_ls is not None and parsed_ls > 0 else None
+
-     liq_l_val = parse_value_to_float(dom_data.get("liquidations_long", 0.0))
-     liq_s_val = parse_value_to_float(dom_data.get("liquidations_short", 0.0))
+     liq_l_val = parse_optional_value(dom_data.get("liquidations_long"))
+     liq_s_val = parse_optional_value(dom_data.get("liquidations_short"))
+
+     def roll_history(cache_key, new_val, default_val):
+         hist = self.obj_cache.get(cache_key)
+         if not hist:
+             seed = default_val if dom_data.get(cache_key) is not None else 0.0
+             hist = [seed] * 120
+         else:
```

```

+         hist = list(hist)[-120:]
+         if len(hist) < 120:
+             pad = hist[0] if hist else (default_val if default_val is not None
else 0.0)
+             hist = [pad] * (120 - len(hist)) + hist
+         if new_val is None:
+             return hist
+         return hist[1:] + [new_val]

        with self._obd_cache_lock:
-         price_history = self.obd_cache.get(f"{symbol_key}_price_history",
[price_val] * 120)
-         price_history = price_history[1:] + [price_val]
+         price_history = _roll_history(f"{symbol_key}_price_history", price_val,
price_val)
        self.obd_cache[f"{symbol_key}_price_history"] = price_history

-         oi_history = self.obd_cache.get(f"{symbol_key}_oi_history", [oi_val] * 120)
-         oi_history = oi_history[1:] + [oi_val]
+         oi_history = _roll_history(f"{symbol_key}_oi_history", oi_val, oi_val)
        self.obd_cache[f"{symbol_key}_oi_history"] = oi_history

-         funding_history = self.obd_cache.get(f"{symbol_key}_funding_history",
[funding_val] * 120)
-         funding_history = funding_history[1:] + [funding_val]
+         funding_history = _roll_history(f"{symbol_key}_funding_history",
funding_val, funding_val)
        self.obd_cache[f"{symbol_key}_funding_history"] = funding_history

-         spot_history = self.obd_cache.get(f"{symbol_key}_{tf}_spot_history",
[spot_cvd_val] * 120)
-         spot_history = spot_history[1:] + [spot_cvd_val]
+         spot_history = _roll_history(f"{symbol_key}_{tf}_spot_history",
spot_cvd_val, spot_cvd_val)
        self.obd_cache[f"{symbol_key}_{tf}_spot_history"] = spot_history

-         fut_history = self.obd_cache.get(f"{symbol_key}_{tf}_fut_history",
[futures_cvd_val] * 120)
-         fut_history = fut_history[1:] + [futures_cvd_val]
+         fut_history = _roll_history(f"{symbol_key}_{tf}_fut_history",
futures_cvd_val, futures_cvd_val)
        self.obd_cache[f"{symbol_key}_{tf}_fut_history"] = fut_history

-         liq_l_history = self.obd_cache.get(f"{symbol_key}_{tf}_liq_l_history",
[liq_l_val] * 120)
-         liq_l_history = liq_l_history[1:] + [liq_l_val]
+         liq_l_history = _roll_history(f"{symbol_key}_{tf}_liq_l_history", liq_l_val,
0.0)
        self.obd_cache[f"{symbol_key}_{tf}_liq_l_history"] = liq_l_history

-         liq_s_history = self.obd_cache.get(f"{symbol_key}_{tf}_liq_s_history",
[liq_s_val] * 120)
-         liq_s_history = liq_s_history[1:] + [liq_s_val]
+         liq_s_history = _roll_history(f"{symbol_key}_{tf}_liq_s_history", liq_s_val,
0.0)
        self.obd_cache[f"{symbol_key}_{tf}_liq_s_history"] = liq_s_history

-         rsi_history = self.obd_cache.get(f"{symbol_key}_{tf}_rsi_history",
[rsi_val] * 120)
-         rsi_history = rsi_history[1:] + [rsi_val]
+         rsi_history = _roll_history(f"{symbol_key}_{tf}_rsi_history", rsi_val,
50.0)
        self.obd_cache[f"{symbol_key}_{tf}_rsi_history"] = rsi_history

-         cb_premium_history = self.obd_cache.get(f"{symbol_key}_cb_premium_history",
[cb_premium_val] * 120)
-         cb_premium_history = cb_premium_history[1:] + [cb_premium_val]
+         cb_premium_history = _roll_history(f"{symbol_key}_cb_premium_history",
cb_premium_val, 0.0)
        self.obd_cache[f"{symbol_key}_cb_premium_history"] = cb_premium_history

-         ls_ratio_history = self.obd_cache.get(f"{symbol_key}_ls_ratio_history",
[ls_val] * 120)
-         ls_ratio_history = ls_ratio_history[1:] + [ls_val]
+         ls_ratio_history = _roll_history(f"{symbol_key}_ls_ratio_history", ls_val,
1.0)
        self.obd_cache[f"{symbol_key}_ls_ratio_history"] = ls_ratio_history

-         live_data[symbol_key] = {
-             "price": price_val,
-             "spot_cvd_15m": spot_cvd_val,
-             "futures_cvd_15m": futures_cvd_val,
-             "funding": funding_val,
-             "oi": oi_val,
-             "premium_index": cb_premium_val,
-             "rsi_15m": rsi_val,
-             "ls_ratio": ls_val,
-             "liquidations_long_15m": liq_l_val,
-             "liquidations_short_15m": liq_s_val,
-         }
+         payload = {}
+         if price_val is not None: payload["price"] = price_val
+         if spot_cvd_val is not None: payload["spot_cvd_15m"] = spot_cvd_val
+         if futures_cvd_val is not None: payload["futures_cvd_15m"] = futures_cvd_val
+         if funding_val is not None: payload["funding"] = funding_val
+         if oi_val is not None: payload["oi"] = oi_val
+         if cb_premium_val is not None: payload["premium_index"] = cb_premium_val
+         if rsi_val is not None: payload["rsi_15m"] = rsi_val
+         if ls_val is not None: payload["ls_ratio"] = ls_val
+         if liq_l_val is not None: payload["liquidations_long_15m"] = liq_l_val
+         if liq_s_val is not None: payload["liquidations_short_15m"] = liq_s_val
+         if payload:
+             live_data[symbol_key] = payload
        return live_data

```

Copy

Patch 4 — Fix LightGBM flag, model reload race, signal cooldown flapping

```

diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
-# Try importing LightGBM (required for predictions)
-try:
-    HAS_LGB = True
-except ImportError:
-    HAS_LGB = False
+
+## Preserve the top-level import result. Do not overwrite HAS_LGB to True when
+## lightgbm failed to import and lgb is None.
+try:
+    HAS_LGB = lgb is not None
+except NameError:
+    HAS_LGB = False
+
+@
+def _maybe_reload_models(self):
+    """Check if models should be reloaded (every 6 hours). Check is inside lock to
prevent TOCTOU race."""
+    with self._model_lock:
+        if time.time() - self._last_model_load > 21600: # 6 hours
+            # Release lock so _load_models can re-acquire it cleanly
+            pass
+        # If reload needed, _load_models acquires its own lock
+        if time.time() - self._last_model_load > 21600:
+            self._load_models()
+    """Reload models every 6 hours."""
+    if time.time() - self._last_model_load <= 21600:
+        return
+    self._load_models()

```

Copy

```

def strategy_eval_worker():
    for symbol in TARGET_ASSETS:
        for strat in strat_by_symbol.get(symbol, []):
            strat_cooldown = strat.get_param(symbol, 'cooldown_seconds',
1800)
            if "TRIGGERED" in strat.state.get(symbol, "") and \
time.time() - strat.last_signal_time.get(symbol, 0) >=
strat_cooldown:
                now_ts = time.time()
                last_ts = strat.last_signal_time.get(symbol, 0)
                if last_ts and (now_ts - last_ts) < strat_cooldown:
                    strat.cooldown_msg[symbol] = f"Cooldown {int(strat_cooldown
- (now_ts - last_ts))}s"
                    continue
                if "TRIGGERED" in strat.state.get(symbol, ""):
                    strat.state[symbol] = "SEARCHING"

```

Patch 5 — Footprint/OBD websocket hardening and non-Binance routing

```

diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
class FootprintScraper:
    def stop(self):
        if loop_to_close is not None:
            try:
                loop_to_close.close()
            except Exception:
                pass
        # The loop is owned by the background thread. Closing it here races
        # with run_until_complete(); let _run_loop close it in its finally.
        self.loop = None

    def _run_loop(self):
        self.loop = asyncio.new_event_loop()
        asyncio.set_event_loop(self.loop)
        self.loop.run_until_complete(asyncio.gather(
            self._listen(),
            self._poll_taker_data(),
            self._compute_profile_metrics()
        ))
        tasks = []
        try:
            tasks = [
                self.loop.create_task(self._listen()),
                self.loop.create_task(self._poll_taker_data()),
                self.loop.create_task(self._compute_profile_metrics()),
            ]
            self.loop.run_until_complete(asyncio.gather(*tasks))
        except RuntimeError as e:
            if self.is_running:
                print(f"[FootprintScraper] Event loop runtime error: {e}")
            finally:
                for t in tasks:
                    if not t.done():
                        t.cancel()
                try:
                    self.loop.run_until_complete(asyncio.gather(*tasks,
return_exceptions=True))
                except Exception:
                    pass
                try:
                    self.loop.close()
                except Exception:
                    pass
@@
    async def _listen(self):
        """aggTrade stream profile only (POC/absorption). Delta/vol authoritative from
        _poll_taker_data REST."""
        if not BINANCE_SYMBOLS:
            while self.is_running:
                await asyncio.sleep(5.0)
            return
        streams = "/".join([f"{s.lower()}@aggTrade" for s in BINANCE_SYMBOLS])
        futures_urls = [
            f"wss://ftstream.binancefuture.com/stream?streams={streams}",
            f"wss://ftstream.binance.com/stream?streams={streams}",
            f"wss://ftstream.binance.me/market/stream?streams={streams}"
        ]
@@
class BinanceOBDWebsocketManager:
    def __init__(self, symbols, data_feed):
        self.symbols = [s.lower() for s in symbols if s.lower().endswith("usdt") or
s.lower().endswith("busd")]
        allowed = set(globals().get("BINANCE_SYMBOLS", symbols))
        self.symbols = [
            s.lower()
            for s in symbols
            if s in allowed and (s.lower().endswith("usdt") or s.lower().endswith("busd"))
        ]
        self.data_feed = data_feed

    streams = "/".join([f"{s}@depth2@100ms" for s in self.symbols])
    streams = "/".join([f"{s}@depth2@100ms" for s in self.symbols])
    self.ws_urls = [
        f"wss://ftstream.binancefuture.com/stream?streams={streams}"
    ]
    self.ws_url = self.ws_urls[0]
    f"wss://ftstream.binance.com/stream?streams={streams}",
    f"wss://ftstream.binance.me/stream?streams={streams}",
    ] if streams else []
    self.ws_url = self.ws_urls[0] if self.ws_urls else ""
@@
    def start(self):
        """Starts the WebSocket manager in a background daemon thread."""
        if not self.ws_urls:
            logger.warning("No Binance OBD symbols configured; OBD websocket disabled.")
            return
        self.running = True

```

Patch 6 — Trade safety: stale/anomaly veto, real costs, peak equity load fix

```

diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
class TradeTracker:
    def __init__(self, log_file="trade_logs.json", initial_capital=INITIAL_CAPITAL):
        self.current_capital = initial_capital
        self.peak_equity = initial_capital
        self.lock = _import_('threading').Lock()
        self.active_trades = {} # key: (symbol, strat_name), val: dict
        self.history = []
        self._load_history()

    self.peak_equity = initial_capital

```

```

        if close and key in self.active_trades:
@@
-         # Calculate PnL percentage and USD
+         # Calculate PnL percentage and USD, including modeled commission
+         # and slippage costs. This is still a paper-fill model, not a
+         # partial-fill ledger.
            units = trade.get("units", 0.0)
            if trade["direction"] == 1:
-             trade["pnl_pct"] = (current_price - trade["entry_price"]) /
trade["entry_price"] * 100
-             trade["pnl_usd"] = (current_price - trade["entry_price"]) * units
+             gross_pnl_usd = (current_price - trade["entry_price"]) * units
+             elif trade["direction"] == -1:
-             trade["pnl_pct"] = (trade["entry_price"] - current_price) /
trade["entry_price"] * 100
-             trade["pnl_usd"] = (trade["entry_price"] - current_price) * units
+             gross_pnl_usd = (trade["entry_price"] - current_price) * units
            else:
-             trade["pnl_pct"] = 0.0
-             trade["pnl_usd"] = 0.0
+             gross_pnl_usd = 0.0
+
+             entry_notional = abs(trade["entry_price"] * units)
+             exit_notional = abs(current_price * units)
+             fees_usd = (entry_notional + exit_notional) * COMMISSION_PCT
+             slippage_usd = (entry_notional + exit_notional) * SLIPPAGE_PCT
+             net_pnl_usd = gross_pnl_usd - fees_usd - slippage_usd
+             basis = entry_notional if entry_notional > 0 else 1.0
+
+             trade["gross_pnl_usd"] = gross_pnl_usd
+             trade["fees_usd"] = fees_usd
+             trade["slippage_usd"] = slippage_usd
+             trade["pnl_usd"] = net_pnl_usd
+             trade["pnl_pct"] = net_pnl_usd / basis * 100.0
@@
        elif direction != 0 and key not in self.active_trades:
@@
            if len(self.active_trades) >= MAX_CONCURRENT_POSITIONS:
                print(f"[PropFirmGuard] Block entry: active trades count
{len(self.active_trades)} >= MAX_CONCURRENT_POSITIONS ({MAX_CONCURRENT_POSITIONS})")
                return False
+
+             feed_data = {}
+             if getattr(self, "feed", None) is not None:
+                 feed_data = self.feed.get_symbol_snapshot(symbol)
+                 if feed_data.get("anomaly_flag"):
+                     print(f"[{symbol}] - {strat_name} Entry blocked: active anomaly
{feed_data.get('anomaly_flag')}")
+                     return False
+                 if hasattr(self.feed, "is_symbol_stale") and self.feed.is_symbol_stale(
+                     symbol,
+                     keys=("price", "futures_cvd_15m", "oi"),
+                     max_age=180.0,
+                 ):
+                     print(f"[{symbol}] - {strat_name} Entry blocked: stale critical feed
data.")
+                     return False
@@
-             feed_data = {}
-             if getattr(self, "feed", None) is not None:
-                 feed_data = self.feed.get_symbol_snapshot(symbol)
+             if not feed_data and getattr(self, "feed", None) is not None:
+                 feed_data = self.feed.get_symbol_snapshot(symbol)

```

Patch 7 — Remove hard-coded credentials and fix shutdown behavior

```

diff --git a/Unified_Engine_3.py b/Unified_Engine_3.py
--- a/Unified_Engine_3.py
+++ b/Unified_Engine_3.py
@@
def ensure_coinglass_logged_in(context):
@@
    email_input = login_page.locator("input[placeholder='email']").first
    if email_input.count() > 0:
        print("[Setup] Login form detected. Performing auto-login...")
-       email = os.environ.get("COINGLASS_EMAIL", "").strip() or
"singhkarabin024@gmail.com"
-       password = os.environ.get("COINGLASS_PASSWORD", "").strip() or "Lufer2hero"
+       email = os.environ.get("COINGLASS_EMAIL", "").strip()
+       password = os.environ.get("COINGLASS_PASSWORD", "").strip()
+       if not email or not password:
+           print("[Setup] Login form detected but COINGLASS_EMAIL/COINGLASS_PASSWORD
are not set. Please login manually or set env vars.")
+           return False
@@
def _graceful_sigint_handler(signum, frame):
@@
-     pass
+     global _SHUTDOWN_REQUESTED
+     if sys.platform == "win32":
+         # Keep Windows behavior: avoid phantom CTRL_C_EVENT from subprocesses.
+         return
+         _SHUTDOWN_REQUESTED = True
@@
        elif ch == b'\x03': # Ctrl+C byte in buffer - treat as quit
            print("\n[Exit] Ctrl+C byte (0x03) detected in keyboard
buffer!")
+             _SHUTDOWN_REQUESTED = True
+             break
            elif ch.lower() == b'q':
                print("\n[Exit] 'q' key pressed!")
+                 _SHUTDOWN_REQUESTED = True
+                 break

```

4. Data / Logic Bugs — P1

P1-1 — AnomalyAgent does not actually halt trading

It writes:

```
self.feed.latest_data[symbol]["anomaly_flag"] = anomaly_detected
```

But TradeTracker previously ignored it. Patch 6 adds an entry veto.

P1-2 — Strategy cooldown is cosmetic without an evaluation skip

Current worker only resets state after cooldown; it does not prevent `evaluate_trigger()` from firing during cooldown. Patch 4 adds a real skip.

P1-3 — OBD websocket includes invalid non-Binance symbols

XAUUSD, XAGUSD, CLUSD, NATGASUSD are explicitly non-Binance but were still included in OBD streams because they end with USD. Patch 5 restricts to `BINANCE_SYMBOLS`.

P1-4 — `parse_value_to_float()` corrupts multi-token values

Current logic removes spaces before trying to split, so values like "123 456" can become "123456". Patch 1 uses regex token extraction.

P1-5 — PnL ignores fees/slippage

The code defines:

COMMISSION_PCT = 0.0004
SLIPPAGE_PCT = 0.0002

Copy

but did not apply them in TradeTracker. Patch 6 applies modeled round-trip costs.

P1-6 — TradeTracker._load_history() peak equity is reset

_load_history() computes peak_equity, then __init__ resets it back to initial capital. Patch 6 fixes that.

5. Concurrency / Thread Safety Findings

Good

- UnifiedDataFeed uses RLock, which is correct because some methods call nested helpers.
- FootprintScraper protects current_candles with self.lock.
- Footprint_store has a dedicated lock.

Needs fixing

1. Nested locks in update_from_footprint()

Previously held feed.lock and footprint_store.lock together. Patch 1 snapshots footprint data first.

2. Playwright page objects should not be used from extra threads

In CoinGlassScraper.get_page_live_data(), this block is risky:

```
threading.Thread(target=inject_layout_indicators, args=(page, 5, 2.0),  
daemon=True).start()
```

Copy

Playwright sync objects are not thread-safe. Prefer inline healing on the same scraper thread or push a heal request into a queue consumed by the Playwright owner thread.

3. FootprintScraper.stop() closes the event loop from the wrong thread

Patch 5 lets the owner thread close its own loop.

6. Architectural Improvements — P2

1. Create a DataQualityGate before strategy evaluation

Centralize checks like:

- price age < 30s
- OI/funding/CVD age < 180s
- no active anomaly
- no recent DOM extraction failure

2. Move source freshness into dashboard/API

Add per-cell age display:

- green <30s
- yellow <180s
- red stale



3. Split monolith into modules again

Minimum split:

- feed.py
- coinglass_scraper.py
- binance_footprint.py
- strategies/alpha_squeezer_v17.py
- risk/trade_tracker.py
- dashboard.py

4. Use queues at thread boundaries

Instead of scrapers directly mutating UnifiedDataFeed, push FeedUpdate objects into a queue.Queue; have one aggregator thread own the feed mutation.

5. Add parser unit tests immediately

Cover

- "1.23K" -> 1230
- "-4.5M" -> -4500000
- "0.01%" -> 0.0001
- "N/A" -> None through parse_optional_value
- "123 456" does not become 123456

6. Cache Gemini meta-label results

Current get_meta_label() can block strategy evaluation for up to 10s per signal. Add a TTL cache by (symbol, direction, regime_bucket).

7. Highest-Priority Next Steps

1. Apply Patch 1, 2, and 3 first. These prevent stale/zero data corruption.
2. Rotate/remove the exposed CoinGlass credential.
3. Apply Patch 6 so anomaly/staleness blocks entries.
4. Run for 30 minutes with --skip-seeding and inspect:
 - latest_dump.json
 - logs/tv_scraper.log
 - stale metric ages via feed.metric_age(symbol, key)

5. Only then enable live/paper trading.

📄 🍏 🗨️ ...

Your credit balance is negative. This run was stopped. Please top up to continue. [upgrade](#)

AI Chat

Why pick ChatGPT or Claude? Both →

Ask anything, create anything

+ GPT-5.5 Pro ▾

